

github.com/githubnext/gpt4-with-calc



Investigating Equipping GPT-4 with Numeric Calculation

Don Syme (dsyme@github.com) + all of GitHub Next esp
Johan and Albert



Equipping for Numeric Calculation

The situation:

- **Any GPT-4 Chat: Context + Question $\Rightarrow$ Answer**



Equipping for Numeric Calculation

The situation:

- **Any GPT-4 Chat: Context + Question $\Rightarrow$ Answer**

The problem:

- GPT-4 is **terrible** at numeric calculations
- It's also terrible at numeric comparisons

*GPT-4 should not be trusted to write a number that is not present verbatim in the input, nor to reason about numbers in any significant way. **In trust scenarios, don't allow GPT-4 to write numbers, and beware that every numeric comparison may be flawed.***



How bad is this?

It's bad



Example fails

- Comparing financial documents
- Example problem:

Both companies reported an increase in net sales compared to the same quarter last year, but Gap Inc. had a much larger absolute and ~~the~~ increase (\$4.04 billion, up 2%) than lululemon (\$1.9 billion, up 28%).



More example problem:

Solving for n , we get:

$$n = \ln(2) / \ln(1 + 0.3)$$

Using a ~~calculator~~ calculator, we get:

$$n \approx 2.53$$



Therefore, it takes about 2.53 years for an item growing at 30% annually to double.



The Hope

The observation:

👎 GPT-4 is **terrible** at **numeric calculations**

👍 GPT-4 is **good** at writing **numeric calculation code**



The Hope made Real

Without numeric calculation equipping:

1. **GPT-4:** Question --> Answer

With numeric calculation equipping:

1. **GPT-4:** Question --> Numeric calculation code
2. **Evaluator:** Numeric calculation code --> Numeric calculation answers
3. **GPT-4:** Question + Numeric calculation code + Numeric calculation answers --> Answer

My opinion: this two-phase technique should be incorporated into **every** GPT-4 request that involves numbers unless explicitly known to be calculation-free. Anything else is



Example question

This is an extract from the Microsoft annual report for 2022:

Microsoft repurchased the following shares of common stock under the share repurchase programs:

(In millions)	Shares	Amount	Shares	Amount	Shares	Amount
Year Ended June 30,		2022		2021		2020
First Quarter	21	\$6,200	25	\$5,270	29	\$4,000
Second Quarter	20	6,233	27	5,750	32	4,600
Third Quarter	26	7,800	25	5,750	37	6,000
Fourth Quarter	28	7,800	24	6,200	28	5,088

What is the overall average and standard deviation of shares repurchased per quarter over these three years in dollars?



Example fail

...lots of working ...

Therefore, the overall average and standard deviation of shares repurchased per quarter over these three years in dollars are \$5 million dollars and 1.16 million dollars, respectively.



Example prompt

```
# Guidance
```

```
Do not answer the question. Instead, write some calculations and comparisons relevant to answering the question. Do this by writing a code block with sections:
```

```
* Definitions
```

```
* Calculations
```

```
* Check
```

```
* Comparisons
```

```
* Return
```



Example prompt

In the optional "Definitions" section:

- * Define a label for each number in the question.
- * Define labels using Javascript 'const'.
- * This section can include valid Javascript single-dimensional arrays.
- * Do not use or create multi-dimensional arrays.
- * Omit this section completely if it contains no definitions.

In the optional "Calculations" section:

- * Define additional relevant labels using Javascript 'const'.
- * Define each label using an expression that references previously defined labels.
- * Do NOT include the calculated values for labels in code or comments.
- * Do NOT solve equations, simply write relevant calculations.
- * Avoid new assumptions in this section. If you make an assumption document it.
- * This section should be valid Javascript definitions.
- * Omit this section completely if it contains no definitions.
- * Use integer division when appropriate.
- * Calculate rounding in code where appropriate.
- * Use values for constants such as pi and e from the Javascript Math library.



Example relevant calculation code

```
# Definitions
shares_2022 = [21, 20, 26, 28] # millions of shares
amount_2022 = [6200, 6233, 7800, 7800] # millions of dollars
shares_2021 = [25, 27, 25, 24] # millions of shares
amount_2021 = [5270, 5750, 5750, 6200] # millions of dollars
shares_2020 = [29, 32, 37, 28] # millions of shares
amount_2020 = [4000, 4600, 6000, 5088] # millions of dollars

# Calculations
total_shares = shares_2022 + shares_2021 + shares_2020 # millions of shares
total_amount = amount_2022 + amount_2021 + amount_2020 # millions of dollars
average_amount = np.mean(total_amount) # millions of dollars
std_amount = np.std(total_amount) # millions of dollars
```



Example relevant results

```
total_shares [21, 20, 26, 28, 25, 27, 25, 24, 29, 32, 37, 28]  
total_amount [6200, 6233, 7800, 7800, 5270, 5750, 5750, 6200, 4000, 4600, 6000, 5088]  
more_shares_2022 False  
more_amount_2022 True  
average_amount 5890.916666666667  
std_amount 1077.077175997874
```

Add code + results to the prompt, before the question



Example answer

The overall average and standard deviation of shares repurchased per quarter over these three years in dollars are:

- Average: \$5,890.92 million
- Standard deviation: \$1,077.08 million

...lots of working ...

Therefore, the overall average and standard deviation of shares repurchased per quarter over these three years in dollars are 17,898.25 million dollars and 1,583.68 million dollars, respectively.



Effective?

Raw calculation (decimals, integers, math functions):

- without equip: **50% mistake rate** (77/153), many extreme mistakes
- with equip: **0% mistake rate** (153/153)

Financial calculations (mortgage, rate of return, WACC):

- without equip: **57% mistake rate**, 4/14
- with equip: **14% mistake rate** 12/14

Tabular data calculations:

- without equip: **50% mistake rate** 4/8
- with equip: **0% mistake rate** 8/8

Unit conversion calculations problems

- without equip: **36% mistake rate** 7/11
- with equip: **0% mistake rate** 11/11

Date calculations:

- without equip: **30% mistake rate** 7/10
- with equip: **20% mistake rate** 8/10

Word puzzles:

- Without equip: **11% mistake rate** (253 failures 2050/2303)
- With equip: **6.2% mistake rate** (145 failures 2158/2303)



Bonus topics

- It's not just **calc**, it's **comparisons** too
- **Units of measure** can be extracted and “tracked”
- Self-**checks**
- **Commenting** your calculations matters



The Devil is in the Detail

Every step of an LLM can introduce hallucination.

Example: **hallucinated numbers in calculation code**



Calculations

```
gap_gross_profit = gap_net_sales * gap_gross_margin # billion USD
lulu_net_revenue_growth = (lulu_net_revenue - 1.5) / 1.5 # fraction
lulu_store_revenue_growth = (lulu_store_revenue - 0.488) / 0.488 # fraction
lulu_ecommerce_revenue_growth = (lulu_ecommerce_revenue - 0.404) / 0.404 # fraction
gap_inventory_growth = (gap_inventory - 2.72) / 2.72 # fraction
lulu_inventory_growth = (lulu_inventory - 0.9) / 0.9 # fraction
gap_capital_expenditures_growth = (gap_capital_expenditures - 308) / 308 # fraction
lulu_capital_expenditures_growth = (lulu_capital_expenditures - 122.5) / 122.5 # fraction
gap_dividend_yield = gap_dividend * 4 / gap_diluted_eps # fraction
lulu_dividend_yield = 0 # fraction
gap_share_repurchases_yield = gap_share_repurchases / gap_net_income # fraction
lulu_share_repurchases_yield = lulu_share_repurchases / lulu_net_income # fraction
```



Example prompt

General guidance for all sections about labels:

- * Use names like ``car_count`` or ``speed_of_car_in_km_per_h`` for each label.
- * Document each label with a descriptive comment at end of line.

General guidance for all sections about units:

- * Every label name should include the unit if known.
- * End each definition with a semicolon.
- * Include a unit at the end of the comment, like [billions_of_dollars], [metric_tons], [square meters], [apples] or [meetings/week].
- * If the unit is unknown use [unknown].

General guidance for all sections about code:

- * Do NOT use lambda expressions, loops, 'if/else', 'return', '=>' or function definitions.
- * Avoid all date/time calculations. Reduce to whole days, hours, minutes and arbitrary seconds.



Example prompt

In the optional "Comparisons" section:

- * Define additional labels relevant to the question using Javascript 'const' by comparing labels using comparison operators and functions and evaluating to single boolean values.
- *  Do NOT include the calculated true/false values for these labels.
- * This section should be valid Javascript definitions.
- * Omit this section completely if it contains no definitions.

In the optional "Check" section:

- * If appropriate, define the label `check\_message` checking if results lie within the known range for the type of number.
- * Omit this section completely if it contains no definitions.
- * If failing the `check\_message` value should evaluate to a string containing a description of why the check failed.
- * If succeeding the `check\_message` value should evaluate to an empty string
- * Do NOT include the calculated string for this label.



TLDR

Calc is hard for AI. Writing calc code is easy

Numeric calc is the “hello world” of trustworthy systems

Numbers are always “rational”. They must always have an explanation.



Chat and Trust

The integration of **Chat**, **Data** and **Programming** is coming very fast. Much faster than I expected even a month ago.

The industry is crying out for AI that can be *trusted*, *explained*, can generate *recognisable*, *analyzable* artifacts like *spreadsheets* and *notebooks*.

By generating calculation code, we have a stepping stone to this.

Important: chat is not just one thing:

- Chat for Entertainment
- Chat for Search
- **Chat for Analytical Question Answering**
- **Chat for Technical Creation**
- **Chat for Tax Returns**



“Code Interpreter” ChatGPT AddIn

Code interpreter Alpha

An experimental ChatGPT model that can use

We p  Ethan Mollick
@emollick

envi This was kind of delightful: I uploaded a CSV file of every lighthouse
location in the US.

eval "ChatGPT Code Interpreter: Create a gif of a map of the lighthouse
locations, where the map is very dark but each lighthouse twinkles." A
uppe couple seconds later...

uplo



II GIF

Boost Your Data Workflow with Noteable and ChatGPT

ChatGPT Plugin for Jupyter Notebooks

Get Started





a brick weighs 30% of a brick plus five GPUs there are 100 bricks and 200 GPUs in a metric ton how much does a brick weigh?

JW

a brick weighs 30% of a brick plus five GPUs there are 100 bricks and 200 GPUs in a metric ton how much does a brick weigh?



Let's call the weight of a brick "B" and the weight of a GPU "G". We can set up an equation based on the information given:

$$B = 0.3 * B + 5 * G$$

We can also set up another equation based on the total weight of 100 bricks and 200 GPUs being equal to 1 metric ton (1000 kilograms):

$$100 * B + 200 * G = 1000$$

We can solve this system of equations to find the weight of a brick (B) and the weight of a GPU (G).

Finished calculating ▾

The weight of a brick is 7.8125 kg.

```
from sympy import symbols, Eq, solve
```

```
B, G = symbols('B G')  
eq1 = Eq(B, 0.3 * B + 5 * G)  
eq2 = Eq(100 * B + 200 * G, 1000)  
solution = solve((eq1, eq2), (B, G))  
solution[B]
```

```
# Result  
7.812500000000000
```



Beyond simple calc

This category of technique can be applied to **any language, computational logic, DSL, query language, library, service** or **computational capability**.

The outputs can be **answers, code, notebooks, proofs, spreadsheets, projects, tests, documentation**.

And tax returns.



Questions for these times

- Hardened, reliable, trustworthy calculation/reasoning systems are **utterly critical**
 - Proof, evidence, reasoning, explanation, audit, provenance and blame are critical in trust domains
- What is “correct” code?
- What establishes trust?
 - What is “evidence” that the chosen code or query is correct?
 - What ontologies, rule/code databases, calc/logic engines help establish trust and confidence?
- How do we document what assumptions have been made?
 - e.g. which calculation formula has been used and what’s a reference for it?





Thank you!

dsyme@github.com

